

The parallel-velocity boundary condition in JOEREK, with the $\mathbf{E} \times \mathbf{B}$ drift term written out

Máté Szűcs

27 August 2026

Abstract

This note writes out, line by line, the Mach-1 (Bohm) boundary condition on the parallel velocity as it is actually implemented in JOEREK's `model600`, including the $\mathbf{E} \times \mathbf{B}$ drift term that JOEREK already carries. Every expression is paired with the exact source line it comes from, in `models/model600/mod_boundary_conditions.f90` at commit `9508effe0` of branch `thermoelectric-ohm`. The last two sections compare the result with the drift-compatible Bohm condition used by SOLPS-ITER and report a measurement, taken with the `diag_mach1` A/B diagnostic, showing that the two differ by exactly one factor of $|\mathbf{B}|$.

1 The velocity representation

`model600` is a reduced-MHD model. The velocity is split into a poloidal $\mathbf{E} \times \mathbf{B}$ part written through a stream function u , and a parallel part written through a variable v_{par} :

$$\mathbf{v} = \underbrace{R \nabla u \times \mathbf{e}_\varphi}_{\mathbf{v}_E} + v_{\text{par}} \mathbf{B}. \quad (1)$$

Two things about this ansatz drive everything below.

(i) The stream-function convention fixes the sign of the potential: $\Phi = -F_0 u$ in `model600`. (The JOEREK reference papers write $\mathbf{v}_{\text{pol}} = -R \nabla u \times \mathbf{e}_\varphi$; the code uses +.)

(ii) v_{par} is *not* the parallel velocity. The physical parallel velocity is

$$v_{\parallel} = v_{\text{par}} |\mathbf{B}| = v_{\text{par}} B_{\text{tot}}.$$

Every factor of $1/B_{\text{tot}}$ in the code exists to undo this. Losing track of it is the single easiest mistake to make in this routine, and §5 is about exactly that.

Writing (1) in components, with $\mathbf{x} = (R, Z)$ poloidal:

$$\mathbf{v}_E = R(-\partial_Z u, \partial_R u), \quad \mathbf{B}_{\text{pol}} = \frac{1}{R}(\partial_Z \psi, -\partial_R \psi), \quad \mathbf{B}_\varphi = \frac{F_0}{R} \mathbf{e}_\varphi. \quad (2)$$

Note the structural symmetry: $\mathbf{v}_E$ is $-R^2$ times the same differential operator applied to u that produces $\mathbf{B}_{\text{pol}}$ from ψ . That symmetry is what makes the identity in §4 come out as cleanly as it does.

The $\mathbf{v}_E$ components appear literally in the diagnostic block, which is the easiest place to read the sign convention off the source:

`mod_boundary_conditions.f90:637-641`

```
u0_x_d = (  Z_t * u0_s_d - Z_s * u0_t_d ) / xjac      ! = du/dR
u0_y_d = ( -R_t * u0_s_d + R_s * u0_t_d ) / xjac      ! = du/dZ
vE_R_d = - BigR * u0_y_d
vE_Z_d = + BigR * u0_x_d
```

2 Geometry at the boundary

The boundary of a bicubic element is a curve of constant s or constant t . The routine sets a single index `iv_dir` that selects the derivative *along* that curve:

`mod_boundary_conditions.f90:254, 259`

```
iv_dir    = 2      ! t_constant_boundary: differentiate w.r.t. s
iv_dir    = 3      ! s_constant_boundary: differentiate w.r.t. t
```

`iv_dir` is the **tangential** derivative index, not the normal one. On a `t_constant` boundary s runs along the wall, so `iv_dir = 2` means $\partial/\partial s$, which is tangential. (`iv_perp_dir`, used elsewhere in the same file, is the normal one.) This is worth stating explicitly because reading it the other way inverts the conclusion of §4.

Throughout, a subscript b denotes this tangential derivative, scaled to the element by `element_size_0`. So for any field f ,

$$f_{0b} \equiv f_{0_b} = \partial_b f = \nabla f \cdot \hat{\mathbf{t}} \Delta\ell, \quad (3)$$

where $\hat{\mathbf{t}}$ is the unit tangent and $\Delta\ell = d\ell = \sqrt{R_b^2 + Z_b^2}$ is the arc length per unit of the local boundary coordinate.

`mod_boundary_conditions.f90:468, 590, 583`

```
ps0_b    = node_list%node(inode)%values(1,iv_dir,var_psi) * element_size_0
u0_b     = node_list%node(inode)%values(1,iv_dir,var_u)   * element_size_0
Vpar0_b  = node_list%node(inode)%values(1,iv_dir,var_Vpar) * element_size_0
```

The remaining geometric quantities:

`mod_boundary_conditions.f90:512-521`

```
normal    = normal / norm2(normal)                                ! outward pointing
direction  = sign(1.d0,dot_product((/ps0_y,-ps0_x/),normal))
Btot      = sqrt(F0**2 + ps0_x**2 + ps0_y**2) / BigR
d1        = sqrt(R_b**2 + Z_b**2)
bn        = dot_product( (/ps0_y,-ps0_x/), normal ) / (BigR*Btot) ! B.n/Btot
```

$$B_{\text{tot}} = \frac{\sqrt{F_0^2 + |\nabla\psi|^2}}{R} = |\mathbf{B}|, \quad b_n = \frac{\mathbf{B} \cdot \hat{\mathbf{n}}}{|\mathbf{B}|} = \frac{\mathbf{B}_{\text{pol}} \cdot \hat{\mathbf{n}}}{B_{\text{tot}}}, \quad \sigma \equiv \text{direction} = \text{sign}(b_n). \quad (4)$$

The toroidal field has no normal component at the wall, so only $\mathbf{B}_{\text{pol}}$ enters b_n . Comparing the `bn` line with (2) gives the sign relation used below,

$$\mathbf{B}_{\text{pol}} \cdot \hat{\mathbf{n}} = b_n B_{\text{tot}}. \quad (5)$$

The obliqueness factor

`factor` is a smoothing weight, active only where the normal field component changes sign between the two ends of a boundary edge (`bn_1 · bn_2 < 0`), i.e. at a tangency point. Away from such points it is identically 1.

`mod_boundary_conditions.f90:534-554`

```

c_1 = vpar_smoothing_coef(1); c_2 = vpar_smoothing_coef(2); c_3 = vpar_smoothing_coef(3)
if ((vpar_smoothing) .and. (bn_1*bn_2 .lt. 0.d0)) then
  if (c_2 .gt. 0d0) then
    factor = 0.25d0 * ( 1.d0 + tanh( (abs(bn) - c_1) / c_2 ) )**2 - c_3
  else
    factor = tanh(bn/c_1)
    direction = 1.d0
  endif
else
  factor = 1.d0
endif
Hfact_b = factor * R_b / BigR + factor_b

```

$$\mathcal{F} \equiv \text{factor} = \begin{cases} \frac{1}{4} \left[1 + \tanh \frac{|b_n| - c_1}{c_2} \right]^2 - c_3, & c_2 > 0, \\ \tanh(b_n/c_1), & \sigma \rightarrow 1, \quad c_2 \leq 0, \\ 1 & \text{away from tangency.} \end{cases} \quad (6)$$

Hfact_b is not a separate physical quantity: it is the logarithmic derivative of the whole prefactor $\mathcal{F}/B_{\text{tot}}$. Under the routine's stated approximation that the field is frozen (so $B_{\text{tot}} \simeq F_0/R$ and $\partial_b B_{\text{tot}}^{-1} = R_b/F_0 = R_b/(R B_{\text{tot}})$),

$$\partial_b \left(\frac{\mathcal{F}}{B_{\text{tot}}} \right) = \frac{1}{B_{\text{tot}}} \underbrace{\left(\mathcal{F} \frac{R_b}{R} + \mathcal{F}_b \right)}_{\text{Hfact_b}}. \quad (7)$$

3 The boundary condition itself

3.1 The value row

mod_boundary_conditions.f90:617 *(the line this note is about)*

```
Mach1BC = - Vpar0 + direction / Btot * factor * cs0 + factor / Btot * BigR**2 * U0_b/ps0_b
```

$$\boxed{\mathcal{M} = -v_{\text{par}} + \frac{\sigma \mathcal{F}}{B_{\text{tot}}} c_s + \frac{\mathcal{F}}{B_{\text{tot}}} R^2 \frac{\partial_b u}{\partial_b \psi} = 0} \quad (8)$$

with the sound speed and its temperature derivatives

mod_boundary_conditions.f90:612-615

```

cs0      = sqrt(gamma*(Ti0+Te0))
cs0_T    = 0.5d0 * gamma / cs0
cs0_TT   = - 0.25d0 * gamma**2 / cs0**3
cs0_TTT  = 3.d0/8.d0 * gamma**3 / cs0**5

```

$$c_s = \sqrt{\gamma(T_i + T_e)}, \quad \frac{\partial c_s}{\partial T} = \frac{\gamma}{2c_s}, \quad \frac{\partial^2 c_s}{\partial T^2} = -\frac{\gamma^2}{4c_s^3}, \quad \frac{\partial^3 c_s}{\partial T^3} = \frac{3\gamma^3}{8c_s^5}. \quad (9)$$

The third term of (8) is the drift term. It is the subject of the next section.

3.2 The tangential-derivative row

The trace space of a bicubic C^1 element carries a value *and* a tangential derivative at each node, so the condition must be imposed on both. Differentiating (8) along the boundary:

`mod_boundary_conditions.f90:621-622, 655`

```
dMach1BC = - Vpar0_b + direction / Btot * factor * cs0_T * (Ti0_b+Te0_b) &
            + direction / Btot * Hfact_b * cs0
! --- only for n_order >= 5:
dMach1BC = dMach1BC + factor / Btot * BigR**2 * U0_bb/ps0_b
```

$$\partial_b \mathcal{M} = -\partial_b v_{\text{par}} + \frac{\sigma \mathcal{F}}{B_{\text{tot}}} \frac{\partial c_s}{\partial T} (\partial_b T_i + \partial_b T_e) + \frac{\sigma}{B_{\text{tot}}} \text{Hfact_b} c_s + \underbrace{\frac{\mathcal{F}}{B_{\text{tot}}} R^2 \frac{\partial_b^2 u}{\partial_b \psi}}_{n_{\text{order}} \geq 5 \text{ only}}. \quad (10)$$

Approximations in (10), stated by the code itself (line 556: “For the BC’s the magnetic field is assumed constant, and we do not take derivatives of psi into account”):

- $\partial_b \psi$, $\partial_b R^2$ and $\partial_b B_{\text{tot}}$ are not differentiated in the drift term. Only R_b/R survives, through `Hfact_b`, and only in the c_s term.
- At $n_{\text{order}} = 3$ — the production setting — **the drift term does not contribute to the derivative row at all**, because $\partial_b^2 u$ is not a degree of freedom. The Bohm condition is drift-corrected in its value but not in its tangential slope.

3.3 The second-derivative row ($n_{\text{order}} \geq 5$)

`mod_boundary_conditions.f90:657-668`

```
d2Mach1BC = - Vpar0_bb + direction / Btot * factor * cs0_TT * (Ti0_b+Te0_b)**2 &
            + direction / Btot * factor * cs0_T * (Ti0_bb+Te0_bb) !E
!+ direction / Btot * Hfact_b * cs0_T * T0_b *2.0 !E
!+ direction / Btot * Hfact_bb * cs0
```

$$\partial_b^2 \mathcal{M} = -\partial_b^2 v_{\text{par}} + \frac{\sigma \mathcal{F}}{B_{\text{tot}}} \left[\frac{\partial^2 c_s}{\partial T^2} (\partial_b T_i + \partial_b T_e)^2 + \frac{\partial c_s}{\partial T} (\partial_b^2 T_i + \partial_b^2 T_e) \right]. \quad (11)$$

The two `Hfact` contributions are commented out in the source, and the drift term has no second-derivative counterpart at all. This row is therefore the least complete of the three; it is unused at $n_{\text{order}} = 3$.

3.4 Jacobian columns

The condition is imposed by row replacement with a large weight `zbig`. The linearisation treats v_{par} , T and u as the only variables — ψ , and with it the whole geometry, is frozen inside a Newton iteration.

`mod_boundary_conditions.f90:618-620, 623-630, 656`

```

Mach1BC_v = - 1.0
Mach1BC_T = + direction / Btot * factor * cs0_T
Mach1BC_u = + factor / Btot * BigR**2 * element_size_0/ps0_b
dMach1BC_v = - element_size_0
dMach1BC_T = + direction / Btot * factor * cs0_TT* T0_b &
+ direction / Btot * Hfact_b * cs0_T
dMach1BC_Tb = + direction / Btot * factor * cs0_T * element_size_0
dMach1BC_ubb= + factor / Btot * BigR**2 * element_size_3/ps0_b

```

row	column	code	expression
value	v_{par}	Mach1BC_v	-1
value	T	Mach1BC_T	$\sigma \mathcal{F} B_{\text{tot}}^{-1} \partial_T c_s$
value	u	Mach1BC_u	$\mathcal{F} B_{\text{tot}}^{-1} R^2 h_0 / \partial_b \psi$
∂_b	v_{par}	dMach1BC_v	$-h_0$
∂_b	T	dMach1BC_T	$\sigma B_{\text{tot}}^{-1} [\mathcal{F} \partial_T^2 c_s \partial_b T + \text{Hfact_b} \partial_T c_s]$
∂_b	$\partial_b T$	dMach1BC_Tb	$\sigma \mathcal{F} B_{\text{tot}}^{-1} \partial_T c_s h_0$

with $h_0 = \text{element_size_0}$. Note that Mach1BC_u is the drift term's Jacobian entry — it already exists, which matters for §7.

4 The identity: what the drift term actually is

The drift term in (8) is written in element coordinates, $R^2 \partial_b u / \partial_b \psi$, which hides its meaning. Converting it costs four lines.

Take the unit tangent oriented as $\hat{\mathbf{t}} = (n_Z, -n_R)$. From (2),

$$\mathbf{v}_E \cdot \hat{\mathbf{n}} = R(-u_Z n_R + u_R n_Z) = R \nabla u \cdot \hat{\mathbf{t}} = R \frac{\partial_b u}{\Delta \ell}, \quad (12)$$

$$\mathbf{B}_{\text{pol}} \cdot \hat{\mathbf{n}} = \frac{1}{R}(\psi_Z n_R - \psi_R n_Z) = -\frac{1}{R} \nabla \psi \cdot \hat{\mathbf{t}} = -\frac{1}{R} \frac{\partial_b \psi}{\Delta \ell}. \quad (13)$$

The arc-length scale $\Delta \ell$ is common to both. Dividing and using (5):

$$R^2 \frac{\partial_b u}{\partial_b \psi} = -\frac{\mathbf{v}_E \cdot \hat{\mathbf{n}}}{\mathbf{B}_{\text{pol}} \cdot \hat{\mathbf{n}}} = -\frac{\mathbf{v}_E \cdot \hat{\mathbf{n}}}{b_n B_{\text{tot}}}. \quad (14)$$

This is why (8) works at all: the ratio of two tangential derivatives of two different fields is a *normal* quantity, because differentiating along the boundary and dotting the corresponding vector into the normal are the same operation up to the same scale factor for both u and ψ . It also explains why the term is written this way: $\partial_b u$ and $\partial_b \psi$ are both directly available as nodal degrees of freedom on the boundary, whereas $\mathbf{v}_E \cdot \hat{\mathbf{n}}$ and b_n are not.

Substituting (14) into (8) and multiplying by B_{tot} to convert $v_{\text{par}} \rightarrow v_{\parallel}$:

$$v_{\parallel}^{\text{JOEK}} = \mathcal{F} \left[\sigma c_s - \frac{\mathbf{v}_E \cdot \hat{\mathbf{n}}}{b_n B_{\text{tot}}} \right] \quad (15)$$

So JOEK's Mach-1 condition is *already* drift-aware: the second term has the $\mathbf{v}_E \cdot \hat{\mathbf{n}}/b_n$ structure of a drift correction, not merely some numerical smoothing.

5 Comparison with the drift-compatible (SOLPS) condition

The Bohm condition constrains the *total* flux into the wall, not the parallel flux. With drifts retained, the total normal velocity is

$$\mathbf{v} \cdot \hat{\mathbf{n}} = v_{\parallel} b_n + \mathbf{v}_E \cdot \hat{\mathbf{n}} \geq \sigma c_s b_n, \quad (16)$$

so the condition to impose on $v_{\parallel}$ is

$$\boxed{v_{\parallel}^{\text{req}} = \sigma c_s - \frac{\mathbf{v}_E \cdot \hat{\mathbf{n}}}{b_n}} \quad (17)$$

which is the form SOLPS-ITER imposes (manual 3.0.9, BCMOM = 3/13), together with a clip at $\pm 2c_s|b_x|$ to keep the correction bounded near tangency.

Comparing (15) with (17), the σc_s terms agree (up to $\mathcal{F}$, which is 1 away from tangency), and the drift terms differ by exactly one factor B_{tot} :

$$\frac{\text{JOEREK drift term}}{\text{required drift term}} = \frac{\mathcal{F}}{B_{\text{tot}}}. \quad (18)$$

Reading. Both terms in (8) carry the same $1/B_{\text{tot}}$ prefactor. That is *correct* for the c_s term, because $v_{\parallel} = v_{\text{par}} B_{\text{tot}}$ converts it into $\mathcal{F} \sigma c_s$. The drift term needed the B_{tot} to survive the same conversion, and it does not have it. This reads as a missing B_{tot} — but that is an inference about intent, not a measurement. The term appears identically in `model1401`, `model1502` and `model1600` with no comment, so it is worth confirming with the authors before changing three models.

6 Measurement

The A/B diagnostic (`models/model1600/mod_mach1_diag.f90`, enabled by `diag_mach1`) accumulates both forms at every boundary node and reports their ratio.

`mod_boundary_conditions.f90:632-650` (*diagnostic only; imposes nothing*)

```
vE_n_d = vE_R_d * normal(1) + vE_Z_d * normal(2)      ! += into the wall
d_jrk_d = factor * BigR**2 * u0_b / ps0_b
call mach1_diag_add(BigR, factor*cs0, d_jrk_d, vE_n_d, vE_t_d, bn, dl)
```

with, inside `mach1_diag_add`, $d_{\text{slp}} = vE_n/|bn|$. By (14) the u -dependence cancels identically and

$$\text{ratio} = \frac{d_{\text{jrk}}}{d_{\text{slp}}} = -\text{sign}(b_n) \frac{\mathcal{F}}{B_{\text{tot}}}, \quad (19)$$

a purely *geometric* quantity. This is a strong prediction: the ratio should be frozen in time even while both drift terms evolve. It is. The measured min, max and mean were byte-identical at every output step over the run.

target	R [m]	measured ratio	R/F_0	agreement
INNER	≈ 1.256	+0.431	0.423	2%
OUTER	≈ 1.594	−0.533	0.536	0.5%

Since $B_{\text{tot}} = \sqrt{F_0^2 + |\nabla\psi|^2}/R \geq F_0/R$, the quantity $1/B_{\text{tot}}$ must sit just *below* R/F_0 — and the outer measurement does exactly that. The opposite signs on the two targets are $\text{sign}(b_n)$ flipping between an entering and an exiting field line, as (19) requires. With $F_0 = 2.9723$, (18) is confirmed to three digits on two independent targets.

Does the difference matter?

Measured at steady state (weak-form residual 9.8×10^{-4} , electrical closure 0.00% on both targets), in units of c_s :

target	JOREK, eq. (15)		required, eq. (17)	
	mean	max	mean	max
INNER	1.36%	115%	3.22%	272%
OUTER	0.25%	46%	0.48%	85%

A shortfall of 1.9% and 0.2% of c_s *in the mean* — but up to 150% of c_s *locally*. The amplifier is grazing incidence:

$$\langle |b_n| \rangle = 0.0080 \text{ (0.46°)}, \quad \min |b_n| = 0.0000 \text{ (tangency)}, \quad (20)$$

and the correction goes as $\mathbf{v}_E \cdot \hat{\mathbf{n}}/b_n$. It is resolvable ($|\mathbf{v}_E \cdot \hat{\mathbf{n}}/b_n| > 10^{-3}c_s$) on only 24% of the inner and 2.3% of the outer boundary weight: concentrated near tangency, not spread over the target. The fraction of boundary weight on which the correction would *reverse* the sign of $v_{\parallel}$ is 0.89%, so SOLPS's $\pm 2c_s|b_x|$ clip is not optional.

7 If it is to be changed

Equation (17) needs the drift term of (8) multiplied by B_{tot} , i.e. the $1/B_{\text{tot}}$ dropped from that term only:

$$\frac{\mathcal{F}}{B_{\text{tot}}} R^2 \frac{\partial_b u}{\partial_b \psi} \longrightarrow \mathcal{F} R^2 \frac{\partial_b u}{\partial_b \psi}, \quad \text{clipped to } |\cdot| \leq 2c_s. \quad (21)$$

The Jacobian entry Mach1BC_u (line 620) scales identically, and dMach1BC_ubb (line 656) likewise — no new columns are needed, because the derivative of the term with respect to u already exists. Put behind a flag (`bohm_drift_compatible`, default `.false.`) the A/B becomes a namelist switch on a single build, and the diagnostic provides its own check: ratio (19) must go to $-\text{sign}(b_n)$ when the flag is on.

Source: `models/model600/mod_boundary_conditions.f90` and `models/model600/mod_mach1_diag.f90`, branch `thermoelectric-ohm`, commit `9508effe0`. Run: AUG #38773, model600, boundary type 1, $n_{\text{tor}} = 1$, weak Galerkin sheath j -V BC active.